Laboratory of Distributed Processing

Laboratory 3

TASK: MULTIPROCESSING WITH SHARED MEMORY AND SEMAPHORES
(use **python** language and **Windows OS**)

1. Create a multiprocessing version of run.py using shared memory constructs.
- Use multiprocessing.Value or multiprocessing.Array.
- Explain what is stored in shared memory. (1 points)

2. Use Semaphore to manage concurrent access.
- Prevent race conditions when updating shared memory.
- Demonstrate multiple processes accessing/modifying shared data. (1 point)

3. Implement structured process communication.
- Properly start, manage, and join processes. (1 point)

4. Add exception handling and timeout mechanisms.
- Implement try/except blocks surrounding shared-memory operations.
- Handle timeouts in tasks if relevant. (1 point)

5. Confirm correct results with the sequential version and compare running times
- Compare running time with the sequential version
- Compare the mean running time and uncertainty with the previous parallel version in laboratory 2. Each parallel should be run 20 times to compute the mean and the uncertainty.
- Uncertainty is computed as: $\Delta T = \frac{T_{max} - T_{min}}{2}$ (1 point)

NOTE: Upload the print-screen of final code execution on enauczanie and of course your code in python :)